

Final Presentation

Architectural Evolution of nopCommerce

Cenário C — Omnichannel Commerce Core

Abel Teixeira · 113655 | Francisco Costa · 131224

Guilherme Pinho · 131690 | Tiago Albuquerque · 112901

Junho 2026 · AS — Arquitetura de Software · UA-DETI

Índice da Apresentação

01 Contexto & Cenário escolhido

02 Análise ao estado atual do nopCommerce

03 Motivações

04 Quality Attribute Scenarios (QA1–QA5)

05 Framework ADD — Desenho arquitetural

06 Arquitetura alvo

07 Implementação realizada

08 ADRs — Decisões arquiteturais

09 Riscos & Evolution Roadmap

10 Feasibility Spike & Runtime Challenge

11 Rastreabilidade

12 Demo & Conclusão

Contexto & Cenário Escolhido

O nopCommerce evolui de plataforma isolada para núcleo de coordenação num ecossistema distribuído de comércio.

Cenário A

Federated Commerce

Aquisições empresariais — múltiplas BUs com base digital partilhada e autonomia local.

Cenário B

Regulated Commerce

Comércio condicionado por regulamentação, validação e auditoria com políticas dinâmicas.

Cenário C

Omnichannel Core ✓

nopCommerce como núcleo central integrando ERP, WMS, Shipping e POS num ecossistema alargado.

Selecionado: Cenário C — evolução para Commerce Coordination Core, integrando sistemas externos de forma desacoplada, resiliente e observável.

Análise ao Estado Atual — nopCommerce

Estrutura por Camadas

- Presentation Layer
- Application / Services Layer
- Domain Layer
- Data Access Layer

Funcionalidades Core

- Catálogo de produtos
- Carrinho & Checkout
- Criação e gestão de encomendas
- Gestão de clientes

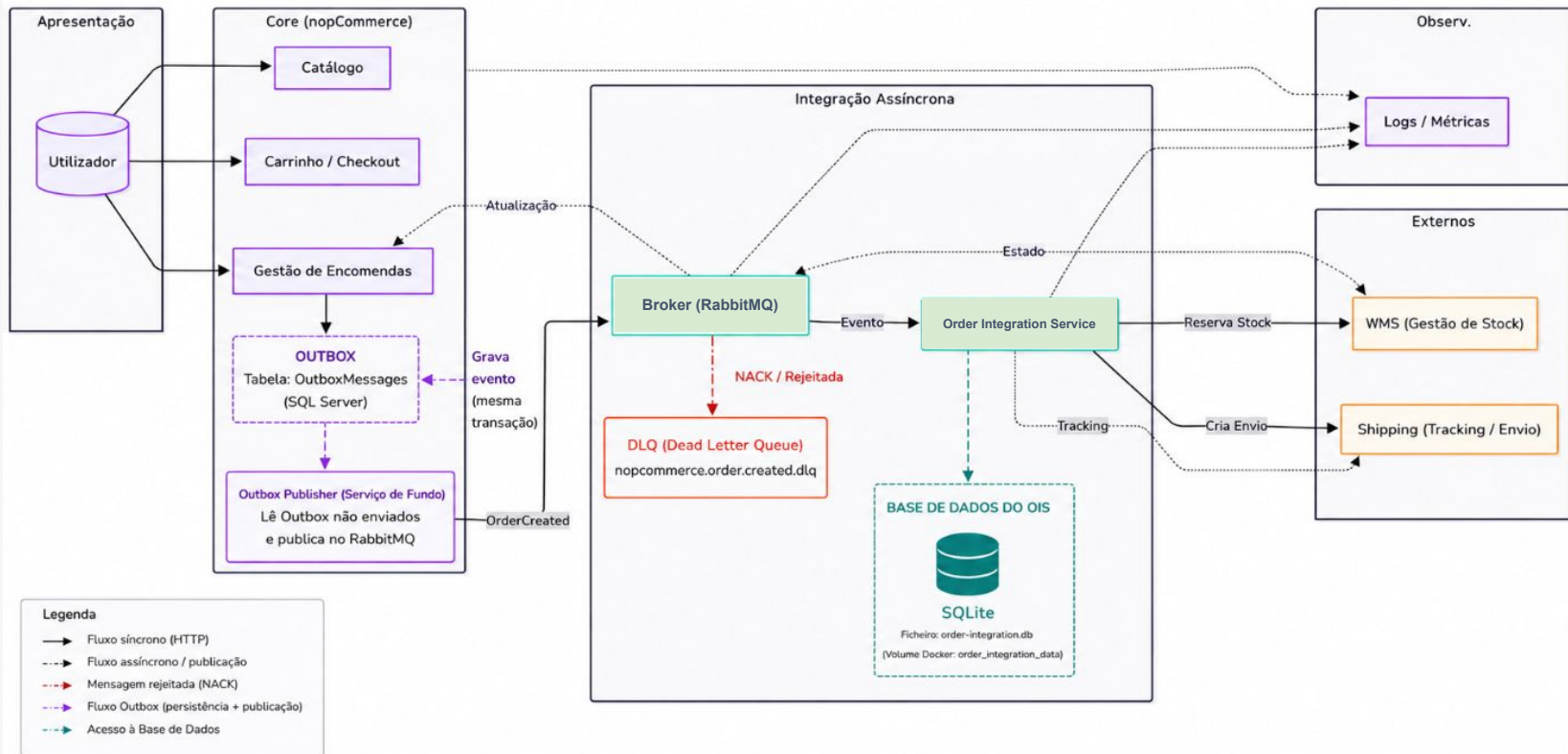
Pontos Fortes

- Centralização do fluxo de negócio
- Modularidade interna por camadas
- Maturidade e estabilidade da plataforma

Limitações

- Dependência de estado interno
- Integração limitada com sistemas externos
- Fluxos predominantemente síncronos
- Acoplamento à lógica central

Arquitetura



Modelação do Domínio — Bounded Contexts

Commerce Experience

Ownership: nopCommerce

Navegação, catálogo, carrinho, checkout, consulta de encomendas.

Warehouse — WMS

Ownership: Sistema externo

Gestão de stock real, reserva de produtos e preparação de encomendas.

Order Management

Ownership: nopCommerce

Criação e gestão do estado lógico das encomendas. Emissão de eventos.

Shipping Service

Ownership: Sistema externo

Processamento de envios, tracking e atualização de estado de entrega.

Order Integration

Ownership: Novo componente

Coordenação entre nopCommerce e sistemas externos. Retries, idempotência, consistência.

WMS e Shipping representados por mocks no Order Integration Service, demonstrando comunicação assíncrona e recuperação de falhas sem sistemas reais.

Motivações

Drivers de Negócio

- Integração com ERP, WMS e Shipping
- Experiência unificada para o cliente
- Continuidade de operação perante falhas externas
- Visibilidade do ciclo completo de encomenda
- Capacidade de integrar novos canais/sistemas futuros

Drivers de Arquitetura

- Resiliência a falhas de sistemas externos
- Consistência eventual entre contextos
- Integração assíncrona via message broker
- Desacoplamento entre componentes
- Observabilidade e diagnóstico distribuído
- Modularidade e extensibilidade incremental

Quality Attribute Scenarios

QA1 — Resiliência a Falhas Externas

Validado

Estímulo: WMS indisponível durante checkout **Resposta:** Evento preservado no RabbitMQ; processamento automático após recuperação

QA2 — Consistência Eventual

Validado

Estímulo: Atraso na atualização do estado de envio **Resposta:** Estado intermédio visível na Integration Status UI; atualizado assincronamente

QA3 — Desacoplamento & Integração

Validado

Estímulo: Integração com novo sistema externo **Resposta:** Order Integration Service independente; nopCommerce não conhece WMS/Shipping

QA4 — Observabilidade & Diagnóstico

Validado

Estímulo: Falha na comunicação com sistema externo **Resposta:** Logs estruturados, RabbitMQ Management UI; tracing distribuído planeado

QA5 — Continuidade de Operação

Validado

Estímulo: Falha externa durante compra **Resposta:** Checkout conclui sem depender do OIS ou WMS; try/catch protege o fluxo

Framework ADD — Attribute-Driven Design

O ADD orienta o desenho a partir dos atributos de qualidade, transformando requisitos não funcionais em decisões arquiteturais concretas. Adequado porque o projeto tem QAs claros e não pretende redesenhar integralmente a plataforma.

Aplicação do ADD — QA → Decisão Arquitetural

QA1 — Resiliência

→ RabbitMQ como Message Broker com Publisher Confirms e DLQ

QA2 — Consist. Eventual

→ Order Integration Service mantém estado externo assíncrono via Integration Status UI

QA3 — Desacoplamento

→ Order Integration Service independente; nopCommerce publica eventos sem conhecer consumidores

QA4 — Observabilidade

→ Logs estruturados + RabbitMQ Management UI; tracing distribuído fase seguinte

QA5 — Continuidade

→ Checkout desacoplado do processamento externo: try/catch protege fluxo principal

Arquitetura Alvo

nopCommerce

Sistema central — catálogo, checkout, encomendas, experiência do utilizador

OrderCreatedEventPublisher

Serializa e publica eventos OrderCreated no RabbitMQ com Publisher Confirms

RabbitMQ (Broker)

Comunicação assíncrona: exchange de tópicos, filas dedicadas e DLQ

Order Integration Service

Consome eventos; coordena WMS e Shipping; gere retries e idempotência

MockStockService (WMS)

Simula reserva de stock — representa Warehouse Management System externo

MockShippingService

Simula criação de envio e geração de tracking code — representa Shipping externo

Integration Status UI

Página HTTP do OIS: estado, stock, shipping, tracking code e timestamps

Data Ownership & Interações

Data Ownership — sem base de dados partilhada

nopCommerce	Catálogo, carrinho, encomendas, estado visível ao cliente
Order Integration Service	Eventos processados, estado de retries, registos de idempotência
WMS	Stock real, reserva de produtos, preparação de encomendas
Shipping Service	Criação de envios, tracking, estado de entrega

Cross-cutting Concerns

- Resiliência: try/catch + Publisher Confirms + DLQ
- Desacoplamento: RabbitMQ desacopla nopCommerce de externos
- Observabilidade: logs + RabbitMQ UI + Integration Status
- Fiabilidade: ACK/NACK + DLQ + filas durable

Interações Síncronas vs Assíncronas

Síncronas

- Navegação/catálogo
- Carrinho e checkout
- Consulta de encomendas

Assíncronas

- Emissão OrderCreatedEvent
- nopCommerce → OIS via broker
- WMS e Shipping updates
- Atualização de estados externos

Implementação — Alterações Realizadas

Evento	OrderCreatedEvent	Contrato de integração com EventId, EventType, OrderId, CustomerId, CreatedAt, Total
Publicação	Publisher no checkout	Após criação da encomenda, publica evento no exchange nopcommerce.order.events
Broker	RabbitMQ integrado	Exchange de tópicos, fila principal, DLQ, definitions.json versionado, docker-compose
OIS	Order Integration Service	Serviço independente em container próprio — consome eventos e coordena WMS/Shipping
WMS mock	MockStockService	Simula reserva de stock; estado passa para Reserved após processamento
Shipping mock	MockShippingService	Simula criação de envio; gera tracking code TRK-{OrderId}-{Code}
Visibilidade	Integration Status UI	Página HTTP com estado, stock, shipping, tracking code e timestamps por encomenda
Fiabilidade	ACK/NACK + DLQ	Mensagem só removida após sucesso; NACK encaminha para DLQ; try/catch no checkout
Readiness	Healthcheck RabbitMQ	nopCommerce e OIS aguardam RabbitMQ saudável antes de arrancar

Fluxo Implementado — Checkout Assíncrono

1

nopCommerce

Utilizador completa checkout no nopCommerce

2

nopCommerce

nopCommerce cria encomenda na base de dados

3

RabbitMQ

OrderCreatedEvent publicado no exchange
nopcommerce.order.events

4

RabbitMQ

Mensagem encaminhada para fila
nopcommerce.order.created

5

OIS

Order Integration Service consome a mensagem → estado:
Received

6

WMS mock

MockStockService reserva stock → estado: Reserved

7

Shipping mock

MockShippingService cria envio e gera tracking code →
estado: Completed

8

OIS

OIS envia ACK ao RabbitMQ → mensagem removida da fila

9

UI

Integration Status UI apresenta encomenda como Completed
com tracking code

ADRs — Architectural Decision Records

ADR 1 Comunicação Assíncrona

Decisão: Introdução de Message Broker para eventos de encomenda

Justificação: Reduz acoplamento; suporta consistência eventual; nopCommerce operacional perante falhas

ADR 2 Order Integration Service

Decisão: Serviço independente para coordenação com externos

Justificação: Separa responsabilidades; permite adicionar/alterar integrações sem tocar no núcleo

ADR 3 Data Ownership Separado

Decisão: Sem base de dados partilhada; comunicação via eventos

Justificação: Desacoplamento e evolução independente por contexto; evita dependências fortes

ADR 4 RabbitMQ como Broker

Decisão: Exchange de tópicos + filas + DLQ + Publisher Confirms

Justificação: Equilíbrio funcionalidade/complexidade; melhor que Kafka (overhead) ou HTTP (síncrono)

ADR 5 Mocks WMS & Shipping

Decisão: MockStockService + MockShippingService no OIS

Justificação: Preserva pressão arquitetural do cenário sem implementar sistemas externos reais

ADR 6 Integration Status UI

Decisão: Página HTTP no OIS com estado externo das encomendas

Justificação: Visibilidade pós-checkout sem alterar profundamente o nopCommerce; demonstra QA2/QA4

Plano de Risco & Estratégias de Mitigação

Fechado	Complexidade comunicação assíncrona P: Médio I: Médio	DLQ + definitions.json versionado + fluxo simplificado (só OrderCreated)
Fechado	Inconsistência temporária de dados P: Alto I: Médio	Contrato de evento definido; estados intermédios visíveis na Integration Status UI
Fechado	Fiabilidade das integrações externas P: Alto I: Alto	Publisher Confirms + DLQ implementados; retry completo depende do mock externo
Fechado	Observabilidade insuficiente P: Médio I: Alto	Logs + RabbitMQ Management UI; dashboards Grafana
Fechado	Broker como ponto único de falha P: Baixo I: Alto	Checkout tolerante a falha (try/catch); filas durable; clustering planeado
Fechado	Evento perdido P: Médio I: Alto	Padrão Outbox - mensagens guardadas em OutboxEvent e publicadas quando o broker está disponível

Evolution Roadmap

Fases 1–3

Infra Assíncrona

Concluído

- Logs e observabilidade base
- RabbitMQ: exchange, filas, DLQ
- Emissão de OrderCreatedEvent

Fases 4–7

Integration Layer

Concluído

- Order Integration Service
- WMS mock (MockStockService)
- Shipping mock (MockShippingService)
- ACK/NACK e DLQ

Fases 8–9

Visibilidade & Resiliência

Concluído

- Integration Status UI
- Operação degradada e recuperação

Fases 10–12

Evolução Futura

Futuro

- Estado externo na UI nopCommerce

Feasibility Spike & Runtime Challenge

Feasibility Spike — Objetivo

Validar a integração assíncrona baseada em eventos entre nopCommerce e sistemas externos.

Resultados Validados

- nopCommerce emite eventos após checkout
- outbox guarda eventos localmente; pública quando o broker está disponível
- RabbitMQ armazena e encaminha eventos
- OIS consome eventos independentemente
- WMS/Shipping mocks processam externamente
- Estado externo visível na Integration Status UI

Runtime Challenge — OIS Indisponível

- 1 OIS parado
- 2 Utilizador faz checkout → encomenda criada
- 3 Evento publicado no RabbitMQ
- 4 **Mensagem fica na fila (Ready: 1)**
- 5 OIS reiniciado
- 6 Mensagem consumida → stock reservado → envio criado
- 7 ACK enviado → fila volta a Ready: 0
- 8 **Integration Status UI: Completed**

Rastreabilidade — Drivers → QA → ADR → Implementação

Driver	Quality Attribute	ADR	Implementação	Estado
Integração com externos	QA1 — Resiliência	ADR 1, ADR 4	RabbitMQ + Publisher	Implementado
Continuidade de operação	QA5 — Continuidade	ADR 1	try/catch no checkout	Implementado
Experiência unificada	QA2 — Consist. Eventual	ADR 3	Data ownership; eventos async	Implementado
Capacidade de evolução	QA3 — Desacoplamento	ADR 2	OIS independente	Implementado
Visibilidade do ciclo	QA4 — Observabilidade	ADR 4	Logs + RabbitMQ UI + DLQ	Implementado
Fiabilidade de mensagens	QA1 — Resiliência	ADR 4	Publisher Confirms + DLQ	Implementado
Separação de dados	Todos os QA	ADR 3	Sem BD partilhada; eventos	Implementado

Demo



Conclusão

- ✓ nopCommerce mantém-se como núcleo da experiência de compra; checkout e criação de encomendas inalterados para o utilizador
- ✓ Mocks de WMS e Shipping preservam a pressão arquitetural sem exigir sistemas reais
- ✓ Comunicação assíncrona com RabbitMQ desacopla o nopCommerce dos sistemas externos (WMS, Shipping)
- ✓ Integration Status UI demonstra consistência eventual e observabilidade pós-checkout
- ✓ Order Integration Service independente coordena o processamento externo e demonstra resiliência e recuperação
- ! Limitações conhecidas: Integration Status, na Admin UI do NopCommerce